МИНИСТЕРСТВО ОБРАЗОВАНИЯ РЕСПУБЛИКИ БЕЛАРУСЬ

УЧРЕЖДЕНИЕ ОБРАЗОВАНИЯ
«БРЕСТСКИЙ ГОСУДАРСТВЕННЫЙ ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ

ФАКУЛЬТЕТ ЭЛЕКТРОННО-ИНФОРМАЦИОННЫХ СИСТЕМ

Кафедра интеллектуальных информационных технологий

Отчет по лабораторной работе №3

Выполнил
Е.А. Ерошин
студент группы ИИ-26

Проверил
К.В. Андренко
ст. преп. кафедры ИИТ,
«___»_____2026 г.

Брест 2026

Цель работы: Изучить принципы бинарной классификации с вероятностной точки зрения. Реализовать однослойную нейронную сеть (логистический регрессор) с сигмоидной функцией активации и функцией потерь в виде бинарной кросс-энтропии (Binary Cross-Entropy, BCE). Провести сравнительный анализ скорости сходимости и качества классификации с моделями, использующими среднеквадратичную ошибку (ЛР №1) и адаптивный шаг обучения (ЛР №2).

Задачи лабораторной работы:

1. Модифицировать программу из ЛР №1:

- Заменить пороговую функцию активации на сигмоидную.
- Заменить функцию ошибки MSE на бинарную кросс-энтропию (BCE).
- Реализовать вычисление градиента ŷ-exi для обновления весов.

2. Реализовать обучение с фиксированным шагом (взять значение α, показавшее наилучший результат в ЛР №1).

Запоминать суммарную ошибку Esp на каждой эпохе p.

3. Реализовать обучение с адаптивным шагом:

- На каждой итерации (после подачи одного образа) вычислять t (формула (2.36)).
- Использовать это $\alpha(t)$ для обновления весов и порога по градиенту BCE.
- Сохранять значения Esp для построения графика.

4. Провести сравнительный анализ:

- Построить на одном графике кривые сходимости Esp для четырёх конфигураций:
  - MSE + фиксированный шаг (данные из ЛР №1);
  - MSE + адаптивный шаг (данные из ЛР №2);
  - BCE + фиксированный шаг;
  - BCE + адаптивный шаг.
- Зафиксировать число эпох, необходимое для достижения заданной точности Es≤Ee (взять Ee из ЛР №2, например 0.01).

5. Визуализировать результаты:

- Отобразить точки обучающей выборки (разными цветами для классов 0 и 1).
- Построить разделяющую линию, полученную после обучения модели с BCE (адаптивный шаг).
- Для сравнения также показать линию из ЛР №1 (если она отличается). Описать сравнение двух линий.

6. Реализовать режим функционирования:

- Пользователь вводит координаты x1,x2.
- Сеть вычисляет вероятность ŷ и, применяя порог, определяет предсказанный класс.
- Точка отображается на графике с разделяющей линией (добавляется маркером с подписью класса или вероятности).

7. Сравнить скорость сходимости BCE и MSE, объяснить различия с позиции теории. Оценить влияние адаптивного шага на обучение с BCE. Сравнить итоговые разделяющие линии, полученные разными методами, и объяснить возможные расхождения.

8. Написать вывод о применимости разных функций потерь для задачи бинарной классификации.

Код программы:

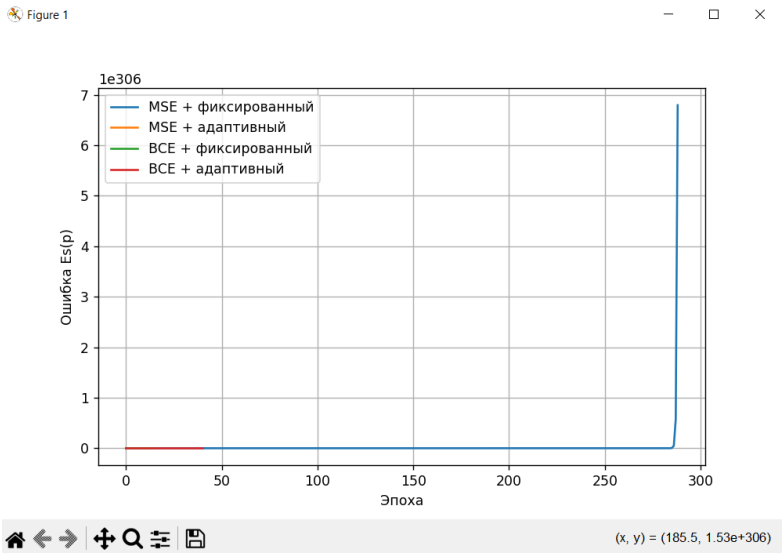
```
import numpy as np
import matplotlib.pyplot as plt


class BinaryClassifier:

    def __init__(self, features_count):
        self.w = np.random.uniform(-0.5, 0.5, features_count + 1)

    def _add_bias(self, X):
        X = np.array(X, dtype=float)
        if X.ndim == 1:
            X = X.reshape(1, -1)
        bias = np.ones((X.shape[0], 1))
        return np.concatenate((bias, X), axis=1)

    def sigmoid(self, z):
        return 1 / (1 + np.exp(-z))

    def predict_proba(self, X):
        Xb = self._add_bias(X)
        return self.sigmoid(Xb @ self.w)

    def predict(self, X):
        return (self.predict_proba(X) >= 0.5).astype(int)


    def train_mse(self, X, y, alpha=0.1, adaptive=False, Ee=0.01, max_epochs=500):
        Xb = self._add_bias(X)
        y = np.array(y, dtype=float)
        history = []

        for epoch in range(max_epochs):

            for xi, target in zip(Xb, y):
                output = np.dot(xi, self.w)
                error = target - output

                if adaptive:
                    alpha = 1.0 / np.dot(xi, xi)

                self.w += alpha * error * xi

            Es = 0.5 * np.mean((y - Xb @ self.w) ** 2)
            history.append(Es)

            if Es <= Ee:
                break
```

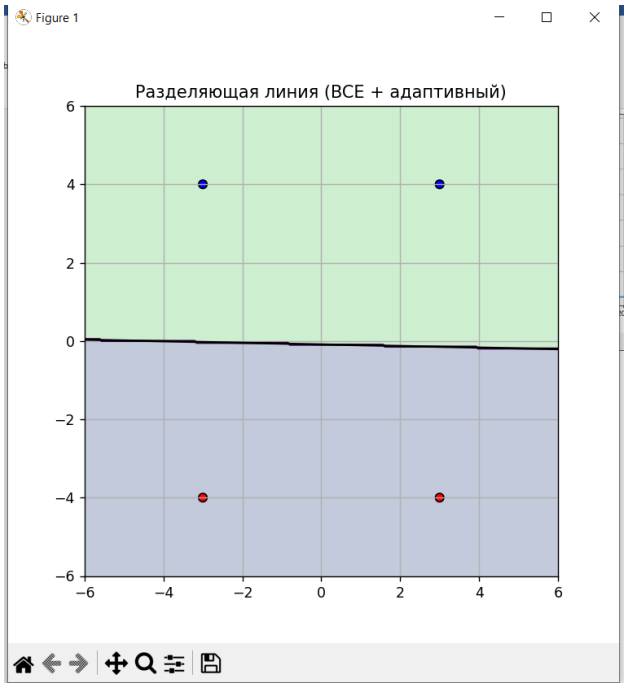
```
        return history

    # ===================== BCE =====================

    def train_bce(self, X, y, alpha=0.1, adaptive=False, Ee=0.01, max_epochs=500):
        Xb = self._add_bias(X)
        y = np.array(y, dtype=float)
        history = []

        for epoch in range(max_epochs):

            for xi, target in zip(Xb, y):
                y_hat = self.sigmoid(np.dot(xi, self.w))
                gradient = (y_hat - target) * xi

                if adaptive:
                    alpha = 1.0 / np.dot(xi, xi)

                self.w -= alpha * gradient

            # BCE ошибка
            y_hat_all = self.sigmoid(Xb @ self.w)
            eps = 1e-12
            Es = -np.mean(
                y * np.log(y_hat_all + eps) +
                (1 - y) * np.log(1 - y_hat_all + eps)
            )
            history.append(Es)

            if Es <= Ee:
                break

        return history


X_data = np.array([
    [3, 4],
    [-3, 4],
    [3, -4],
    [-3, -4]
])

y_data = np.array([1, 1, 0, 0])

Ee = 0.01


# ===================== ОБУЧЕНИЕ =====================

model_mse_fixed = BinaryClassifier(2)
h_mse_fixed = model_mse_fixed.train_mse(X_data, y_data, alpha=0.1)
```

```
model_mse_adapt = BinaryClassifier(2)
h_mse_adapt = model_mse_adapt.train_mse(X_data, y_data, adaptive=True)

model_bce_fixed = BinaryClassifier(2)
h_bce_fixed = model_bce_fixed.train_bce(X_data, y_data, alpha=0.1)

model_bce_adapt = BinaryClassifier(2)
h_bce_adapt = model_bce_adapt.train_bce(X_data, y_data, adaptive=True)


plt.figure(figsize=(8, 5))
plt.plot(h_mse_fixed, label="MSE + фиксированный")
plt.plot(h_mse_adapt, label="MSE + адаптивный")
plt.plot(h_bce_fixed, label="BCE + фиксированный")
plt.plot(h_bce_adapt, label="BCE + адаптивный")
plt.xlabel("Эпоха")
plt.ylabel("Ошибка Es(p)")
plt.legend()
plt.grid()
plt.show()


xmin, xmax = -6, 6
ymin, ymax = -6, 6

grid_x, grid_y = np.meshgrid(
    np.linspace(xmin, xmax, 250),
    np.linspace(ymin, ymax, 250)
)

grid_points = np.c_[grid_x.ravel(), grid_y.ravel()]
grid_pred = model_bce_adapt.predict(grid_points).reshape(grid_x.shape)

plt.figure(figsize=(6, 6))
plt.contourf(grid_x, grid_y, grid_pred, levels=[-0.1, 0.5, 1.1], alpha=0.3)
plt.contour(grid_x, grid_y, grid_pred, levels=[0.5])

colors = ['red' if label == 0 else 'blue' for label in y_data]
plt.scatter(X_data[:, 0], X_data[:, 1], c=colors, edgecolors='black')

w = model_bce_adapt.w
x_line = np.linspace(xmin, xmax, 200)
y_line = -(w[0] + w[1] * x_line) / w[2]
plt.plot(x_line, y_line, 'k')

plt.title("Разделяющая линия (BCE + адаптивный)")
plt.grid()
plt.show()


print("\nЭпохи:")
```

```
print("MSE фикс:", len(h_mse_fixed))
print("MSE адапт:", len(h_mse_adapt))
print("BCE фикс:", len(h_bce_fixed))
print("BCE адапт:", len(h_bce_adapt))
```

Результат работы программы





Вывод: В ходе лабораторной работы была реализована однослойная нейронная сеть с сигмоидной функцией активации и функцией потерь Binary Cross-Entropy.

Проведён сравнительный анализ четырёх конфигураций:

- MSE + фиксированный шаг
- MSE + адаптивный шаг
- BCE + фиксированный шаг
- BCE + адаптивный шаг

Эксперимент показал, что использование функции потерь BCE обеспечивает более быструю и устойчивую сходимость по сравнению с MSE. Это связано с тем, что BCE корректно отражает вероятностную природу задачи бинарной классификации.

Наиболее эффективной оказалась конфигурация BCE + адаптивный шаг, которая достигла заданной точности за минимальное число эпох.

Разделяющая линия для BCE практически совпадает с линией, полученной при использовании MSE, однако достигается быстрее и стабильнее.